

Projeto SaaS WaaS: Plataforma de Disparos e CRM para WhatsApp

Este documento detalha a arquitetura, stack tecnológica, funcionalidades competitivas e o plano de execução para a criação de um SaaS de disparos em massa e gestão de WhatsApp, otimizado para rodar em um servidor VPS com gerenciamento via Coolify.

1. Infraestrutura e Integração (Hostinger + Hostgator)

A base do projeto utilizará o servidor atual (VPS KVM 2, 8GB RAM, 2 vCPU). A gestão de deploy será feita inteiramente pelo **Coolify**, que orquestrará os containers Docker.

O Coolify interceptará esses subdomínios, aplicará o certificado SSL gratuito (Let's Encrypt) via Traefik/Caddy e direcionará para os containers corretos.

2. Escolha da Tech Stack

Para um sistema de alta concorrência como envio de mensagens, a stack precisa ser assíncrona e orientada a eventos.

Camada	Tecnologia Recomendada	Justificativa / Papel no Projeto
Motor Core (WhatsApp)	Evolution API v2 (Node.js/Baileys)	Não reinvente a roda criando a conexão socket do zero. A Evolution API é um microserviço open-source robusto que

Camada	Tecnologia Recomendada	Justificativa / Papel no Projeto
		gerencia múltiplas instâncias, Webhooks, filas de envio e reconexões automáticas. Ela servirá como o "cérebro" do WhatsApp.
Backend / API (SaaS)	Node.js com NestJS (TypeScript)	Arquitetura modular, excelente para SaaS multitenant. Gerenciará os usuários, assinaturas, lógica das campanhas, upload de planilhas e comunicação com a Evolution API.
Frontend / Painei	Next.js (React) + Tailwind CSS + Shadcn UI	Next.js garante performance. Tailwind e Shadcn UI permitem criar interfaces complexas (tabelas, modais, dashboards modernos com visual clean/glassmorphism) rapidamente.
Banco de Dados Core	PostgreSQL	Relacional e confiável. Essencial para garantir a integridade dos dados de usuários (SaaS), faturas, tabelas de contatos, histórico de campanhas e

Camada	Tecnologia Recomendada	Justificativa / Papel no Projeto
		tags.
Fila e Cache (Crucial)	Redis + BullMQ	Obrigatório. Disparos em massa não podem travar a API principal. O Redis criará filas de espera. Se uma campanha tem 10.000 números, o BullMQ processa em background respeitando limites de tempo para evitar banimentos.

3. Funcionalidades para Competir no Mercado (Escalabilidade)

Para que o SaaS tenha valor de mercado real frente aos concorrentes, ele não pode apenas enviar mensagens. Ele deve proteger o cliente de bloqueios e personalizar a experiência.

Funcionalidades MVP (Requisitadas)

- **Gestão de Múltiplas Instâncias:** Leitura de QR Code diretamente no painel para conectar vários números.
- **Importação Inteligente:** Upload de CSV/XLSX. O sistema deve mapear as colunas automaticamente (ex: "A coluna 1 é o Nome, a 2 é o Telefone").
- **Gestão de Contatos:** Higienização automática (remover duplicações, formatar números no padrão DDI+DDD). Separação por Tags/Listas.
- **Construtor de Campanhas:** Agendamento por data/hora, seleção de instâncias remetentes e anexos (imagens, PDFs, áudio gravado na hora).

Funcionalidades Avançadas (Diferenciais Competitivos)

- **Sistema Anti-Banimento (Spintax):** Capacidade de rotacionar palavras. Exemplo: {Olá|Oi|Tudo bem|Como vai} {Nome}. O sistema envia uma variação diferente para cada lead, enganando o algoritmo de spam do WhatsApp.
- **Delay Humanizado:** Configuração de pausa randômica entre envios. Ex: Enviar mensagem 1, aguardar de 15 a 45 segundos aleatoriamente, enviar mensagem 2.
- **Áudio como "Gravado na Hora":** Envio de áudios em formato .ogg com metadata específica (PTT) para que apareça como "gravando..." e não como áudio encaminhado.
- **Warm-up (Aquecimento de Chip):** Módulo onde o usuário conecta 3 ou 4 chips recém-comprados, e o sistema faz com que eles conversem entre si usando inteligência artificial ou scripts pré-programados para gerar histórico antes de disparos em massa.
- **Multitenancy (SaaS):** Planos limitadores. Exemplo: Plano Basic (1 instância, 1000 envios/mês), Plano Pro (5 instâncias, envios ilimitados). Integração com Stripe ou Mercado Pago para cobrança recorrente.

4. Arquitetura Lógica do Sistema

O fluxo de uma campanha ocorrerá da seguinte forma para otimizar o uso do seu servidor:

1. O usuário entra no **Painel (Next.js)** e faz o upload de uma planilha.
2. O **Backend (NestJS)** recebe o arquivo, limpa as duplicações, salva os leads no **PostgreSQL** e vincula a uma campanha.
3. O usuário agenda o disparo. O Backend cria um "Job" na fila do **Redis (BullMQ)** para a hora marcada.
4. No momento do disparo, o "Worker" do backend retira os números da fila aos poucos (respeitando o delay configurado), processa as variáveis Spintax (troca [Nome] pelo nome real) e faz chamadas HTTP para a **Evolution API**.
5. A **Evolution API** se comunica com os servidores do WhatsApp e envia as mensagens. Retorna via Webhook se a mensagem foi Entregue, Lida ou Falhou.
6. O Backend recebe o Webhook e atualiza os gráficos no painel em tempo real via WebSocket.

5. Roadmap de Desenvolvimento (Tasks Iniciais)

Fase	Tarefas (Tasks)	Complexidade
Fase 1: Infraestrutura	1. Configurar Coolify no VPS. 2. Apontar DNS Hostgator. 3. Fazer deploy do PostgreSQL e Redis pelo Coolify. 4. Fazer deploy da Evolution API (imagem Docker) via Coolify.	Baixa / Média
Fase 2: Backend Core	1. Criar Auth (Login/Registro/JWT). 2. Criar CRUD de Instâncias (Integrar endpoint de gerar QR Code da Evolution API). 3. Criar lógica de Planilhas (Parsing de CSV/Excel) e salvar em banco. 4. Implementar serviço de fila BullMQ.	Alta
Fase 3: Frontend Painel	1. Criar Layout Base (Sidebar, Header). 2. Tela de Instâncias (Exibir QR Code e Status). 3. Tela de Contatos/Tags (Datatables). 4. Tela de Criação de Campanha (Stepper: Upload -> Mensagem ->	Média

Fase	Tarefas (Tasks)	Complexidade
	Agendamento).	
Fase 4: Motor de Disparo	1. Lógica Worker: Pegar contatos da fila. 2. Implementar substituição de variáveis e Spintax. 3. Implementar Delay randômico. 4. Lidar com Webhooks de retorno de status (Enviado/Lido).	Alta
Fase 5: SaaS & Monetização	1. Criar tabelas de Planos e Assinaturas. 2. Integração com gateway de pagamento. 3. Bloqueios de uso por limite de plano.	Média

Considerações sobre a Hospedagem atual: O servidor de 8GB RAM suportará tranquilamente o início do projeto. A Evolution API consome cerca de 100MB a 200MB de RAM por instância conectada de WhatsApp. Você conseguirá alocar confortavelmente entre 15 a 20 instâncias simultâneas de clientes antes de precisar fazer um upgrade no painel da Hostinger para um plano superior.